

Smart Academic Advisor - Code Trace

Grad project discussion PDF: the most important source snippets from login through protected routing, data loading, advisor transcript review, admin mark editing, API persistence, and SQL-derived GPA views.

Generated from repository source on 2026-05-15.

Primary defense points

- Authentication is handled by Supabase Auth, but app roles come from public.app_users and are checked before role routes are shown.
- The login flow is protected by reCAPTCHA before AuthContext.login processes credentials.
- Advisor views are read-only and scoped to the logged-in advisor via advisorId.
- Changing marks is admin-only: route guard in App.tsx, UI validation in AdminStudentTranscriptPage, and token/admin verification in the server endpoint.
- Transcript marks are persisted in student_transcript_entries; SQL views recompute transcript rows, GPA, and completed credits for dashboards.

End-to-end trace

- 1. User opens /login, selects role, enters ID and password.
- 2. LoginPage obtains and verifies a reCAPTCHA token before credentials are processed.
- 3. AuthContext.login looks up the app user by university ID, checks selected role/status, and signs in with Supabase email/password.
- 4. AuthContext resolves the Supabase auth user back to public.app_users and stores the role-aware session.
- 5. App.tsx routes the user to the correct protected area. Admin transcript editing is only reachable through an admin route.
- 6. AppDataContext loads Supabase views into frontend state: courses, students, transcripts, GPA summaries, drafts, and evaluations.
- 7. Advisors inspect advisee transcript and risk views, guarded by advisorId === current user ID.
- 8. Admins open a student transcript, edit mark/attempt/term fields, and click Save changes.
- 9. The admin transcript page validates the edits and calls upsertTranscriptEntry for each changed row.
- 10. upsertTranscriptEntry revalidates, posts to /api/admin-upsert-transcript-entry, then refreshes derived frontend transcript/GPA state.
- 11. The server endpoint checks the bearer token belongs to an admin and writes student_transcript_entries using the service role.
- 12. SQL views expose updated transcript rows and recomputed GPA/completed credits back to the dashboards.

1. Login Form: role, reCAPTCHA, credentials

Source: src/pages/LoginPage.tsx:20-93

- Shows the role selector state and login submission entry point.
- Checks Supabase config, verifies reCAPTCHA, calls AuthContext.login, then routes to the selected role home.

Code snippet

```
20 export default function LoginPage() {
21   const { isAuthenticated, login, user } = useAuth();
22   const { submitPasswordResetInquiry } = useAppData();
23   const navigate = useNavigate();
```

```

24  const location = useLocation();
25  const authError = (location.state as { authError?: string } | null)?.authError ?? null;
26
27  const [selectedRole, setSelectedRole] = useState<Role>('student');
28  const [userId, setUserId] = useState('');
29  const [password, setPassword] = useState('');
30  const [rememberMe, setRememberMe] = useState(false);
31  const [isSubmitting, setIsSubmitting] = useState(false);
32  const [error, setError] = useState<string | null>(null);
33  const [isForgotPasswordOpen, setIsForgotPasswordOpen] = useState(false);
34  const [forgotPasswordId, setForgotPasswordId] = useState('');
35  const [forgotPasswordMessage, setForgotPasswordMessage] = useState<string | null>(null);
36  const [forgotPasswordError, setForgotPasswordError] = useState<string | null>(null);
37  const [isSubmittingInquiry, setIsSubmittingInquiry] = useState(false);
38
39  useEffect(() => {
40    if (isAuthenticated && user) {
41      navigate(getHomeRoute(user.role), { replace: true });
42    }
43  }, [isAuthenticated, navigate, user]);
44
45  useEffect(() => {
46    if (authError) {
47      navigate(location.pathname, { replace: true, state: null });
48    }
49  }, [authError, location.pathname, navigate]);
50
51  const handleRoleChange = (role: Role) => {
52    setSelectedRole(role);
53    setError(null);
54    setForgotPasswordError(null);
55    setForgotPasswordMessage(null);
56    if (role === 'admin') {
57      setIsForgotPasswordOpen(false);
58    }
59  };
60
61  const handleSubmit = async (event: React.FormEvent) => {
62    event.preventDefault();
63    setError(null);
64
65    if (backendConfigError) {
66      setError(backendConfigError);
67      return;
68    }
69
70    if (!hasRecaptchaSiteKey()) {
71      setError('reCAPTCHA is not configured yet. Add the site key before logging in.');
```

```

84     return;
85   }
86
87   navigate(getHomeRoute(selectedRole), { replace: true });
88 } catch (submissionError) {
89   setError(submissionError instanceof Error ? submissionError.message : 'Unable to verify reCAPTCHA.');
```

2. reCAPTCHA Client Verification Call

Source: src/lib/recaptcha.ts:80-139

- Generates the browser token and posts it to the server verification endpoint.
- Keeps login protected before credentials are sent to Supabase.

Code snippet

```

80 export function hasRecaptchaSiteKey() {
81   return Boolean(RECAPTCHA_SITE_KEY);
82 }
83
84 export async function executeRecaptcha(action: string) {
85   if (!RECAPTCHA_SITE_KEY) {
86     throw new Error('reCAPTCHA is not configured for this environment.');
```

```

122     });
123
124     const payload = (await response.json()) as RecaptchaVerificationResult;
125     if (!response.ok || !payload.success) {
126         throw new Error(payload.error ?? 'reCAPTCHA verification failed.');
```

3. reCAPTCHA Server Endpoint

Source: `api/verify-recaptcha.ts:45-145`

- Rejects non-POST requests and missing tokens.
- Bypasses preview deployments, but validates production requests against Google with action and score checks.

Code snippet

```

45 export default async function handler(request: VercelRequest, response: VercelResponse) {
46     if (request.method !== 'POST') {
47         response.status(405).json({ success: false, error: 'Method not allowed.' });
48         return;
49     }
50
51     const secret = process.env.RECAPTCHA_SECRET_KEY;
52     const isPreviewDeployment = isTestingDeployment(request);
53     if (!secret) {
54         response.status(500).json({ success: false, error: 'reCAPTCHA secret key is not configured.' });
55         return;
56     }
57
58     const payload = typeof request.body === 'string'
59         ? JSON.parse(request.body) as { token?: string; action?: string }
60         : (request.body ?? {}) as { token?: string; action?: string };
61
62     if (!payload.token || !payload.action) {
63         response.status(400).json({ success: false, error: 'Missing reCAPTCHA token or action.' });
64         return;
65     }
66
67     if (isPreviewDeployment) {
68         response.status(200).json({
69             success: true,
70             score: 1,
71             action: payload.action,
72             bypassed: true,
73         });
74         return;
75     }
76
77     const verificationBody = new URLSearchParams({
78         secret,
```

```

79     response: payload.token,
80   });
81
82   try {
83     const googleResult = await Promise.race([
84       fetch(RECAPTCHA_VERIFY_URL, {
85         method: 'POST',
86         headers: {
87           'Content-Type': 'application/x-www-form-urlencoded',
88         },
89         body: verificationBody.toString(),
90       }).then(async (googleResponse) => {
91         if (!googleResponse.ok) {
92           throw new Error('Google verification failed with status ${googleResponse.status}.');
93         }
94
95         return (await googleResponse.json()) as {
96           success?: boolean;
97           score?: number;
98           action?: string;
99           ['error-codes']?: string[];
100         };
101       }),
102       timeoutAfter(VERIFY_TIMEOUT_MS, {
103         success: false,
104         score: 0,
105         action: payload.action,
106         ['error-codes']: ['verification-timeout'],
107       }),
108     ]);
109
110     const score = Number(googleResult.score ?? 0);
111     const expectedAction = normalizeAction(payload.action);
112     const returnedAction = normalizeAction(googleResult.action);
113     const hasReturnedAction = returnedAction.length > 0;
114     const isActionMatch = !hasReturnedAction || returnedAction === expectedAction;
115     const isVerified = Boolean(googleResult.success) && isActionMatch && score >= MIN_RECAPTCHA_SCORE;
116
117     if (!isVerified) {
118       const timeoutHit = googleResult['error-codes']?.includes('verification-timeout');
119
120       response.status(timeoutHit ? 504 : 400).json({
121         success: false,
122         score,
123         action: googleResult.action,
124         errors: googleResult['error-codes'] ?? [],
125         error: timeoutHit
126           ? 'Timed out while talking to Google reCAPTCHA.'
127           : !isActionMatch
128             ? 'reCAPTCHA action mismatch.'
129             : score < MIN_RECAPTCHA_SCORE
130               ? 'Suspicious activity detected. Please try again.'
131               : 'reCAPTCHA verification failed.',
132       });
133       return;
134     }
135
136     response.status(200).json({
137       success: true,
138       score,

```

```

139     action: googleResult.action,
140   });
141 } catch (error) {
142   const message = error instanceof Error ? error.message : 'Unable to verify reCAPTCHA right now.';
143   response.status(502).json({ success: false, error: message });
144 }
145 }

```

4. Auth Session Hydration and Supabase User Mapping

Source: `src/context/AuthContext.tsx:481-650`

- Maps Supabase auth users back to `public.app_users` records.
- Restores session state on refresh and listens for Supabase auth changes.

Code snippet

```

481 const resolveAppUserFromAuth = useCallback(async (authUser: User | null) => {
482   if (!authUser || !hasSupabaseConfig()) {
483     return null;
484   }
485
486   const authUserId = authUser.id;
487   const encodedAuthUserId = encodeURIComponent(authUserId);
488   let remoteRows = await supabaseSelect<RemoteAppUserRow[]>(
489     'app_users',
490     'select=id,auth_user_id,university_id,role,full_name,initials,email,subtitle,status,last_login_at,last_seen_at&auth_user_id=eq.${encodedAuthUserId}&limit=1'
491   );
492
493   if (remoteRows.length === 0 && authUser.email) {
494     const encodedEmail = encodeURIComponent(authUser.email);
495     remoteRows = await supabaseSelect<RemoteAppUserRow[]>(
496       'app_users',
497       'select=id,auth_user_id,university_id,role,full_name,initials,email,subtitle,status,last_login_at,last_seen_at&email=eq.${encodedEmail}&limit=1'
498     );
499
500     if (remoteRows.length > 0 && remoteRows[0]?.auth_user_id !== authUserId) {
501       await supabasePatch<RemoteAppUserRow[]>(
502         'app_users',
503         'id=eq.${encodeURIComponent(remoteRows[0].id)}',
504         { auth_user_id: authUserId }
505       );
506       remoteRows = [{ ...remoteRows[0], auth_user_id: authUserId }];
507     }
508   }
509
510   const resolved = remoteRows[0];
511   if (!resolved) {
512     return null;
513   }
514
515   const passwordById = new Map(usersRef.current.map((account) => [account.id, account.password]));
516   const mapped = mapRemoteUser(resolved, passwordById);
517
518   setUsers((current) => {
519     const nextUsers = mergeManagedUsers(SEED_MANAGED_USERS.map(normalizeManagedUser), current, [mapped]).map(normalizeManagedUser);
520     return areManagedUsersEqual(current, nextUsers) ? current : nextUsers;
521   });
522
523   return toSessionUser(mapped);
524 }, []);
525

```

```

526 useEffect(() => {
527   if (hasSupabaseConfig()) {
528     window.localStorage.removeItem(USERS_KEY);
529     return;
530   }
531
532   window.localStorage.setItem(USERS_KEY, JSON.stringify(users.map(normalizeManagedUser)));
533 }, [users]);
534
535 useEffect(() => {
536   if (!isLocalDemoModeEnabled()) {
537     window.localStorage.removeItem(LOCAL_SESSION_KEY);
538     window.sessionStorage.removeItem(SESSION_SESSION_KEY);
539     window.localStorage.removeItem(REMEMBER_ME_KEY);
540     return;
541   }
542
543   window.localStorage.removeItem(LOCAL_SESSION_KEY);
544   window.sessionStorage.removeItem(SESSION_SESSION_KEY);
545
546   if (user) {
547     const target = rememberSession ? window.localStorage : window.sessionStorage;
548     target.setItem(rememberSession ? LOCAL_SESSION_KEY : SESSION_SESSION_KEY, JSON.stringify(user));
549     window.localStorage.setItem(REMEMBER_ME_KEY, rememberSession ? 'true' : 'false');
550   } else {
551     window.localStorage.removeItem(REMEMBER_ME_KEY);
552   }
553 }, [rememberSession, user]);
554
555 useEffect(() => {
556   window.localStorage.setItem(ATTEMPTS_KEY, JSON.stringify(attempts));
557 }, [attempts]);
558
559 useEffect(() => {
560   if (!hasSupabaseConfig()) {
561     return;
562   }
563
564   let cancelled = false;
565
566   const hydrate = async () => {
567     try {
568       await syncUsersFromSupabase();
569       const {
570         data: { session },
571       } = await getSupabaseSession();
572
573       if (cancelled) {
574         return;
575       }
576
577       if (!session?.user) {
578         setUser(null);
579         setIsAuthReady(true);
580         return;
581       }
582
583       const resolved = await resolveAppUserFromAuth(session.user);
584       if (cancelled) {
585         return;

```

```

586     }
587
588     setUser(resolved);
589     setIsAuthReady(true);
590   } catch (error) {
591     console.error('Unable to initialize Supabase auth session.', error);
592     if (!cancelled) {
593       setUser(null);
594       setIsAuthReady(true);
595     }
596   }
597 };
598
599 void hydrate();
600
601 const { data: authListener } = onSupabaseAuthStateChange((event, session) => {
602   if (cancelled) return;
603
604   if (event === 'SIGNED_OUT') {
605     setUser(null);
606     setIsAuthReady(true);
607     return;
608   }
609
610   // Ignore transient null sessions (e.g. during TOKEN_REFRESHED)
611   if (!session?.user) {
612     return;
613   }
614
615   if (event === 'SIGNED_IN' || event === 'TOKEN_REFRESHED') {
616     void (async () => {
617       if (cancelled) return;
618       const resolved = await resolveAppUserFromAuth(session.user);
619       if (cancelled) return;
620       if (resolved) {
621         setUser(resolved);
622       }
623       setIsAuthReady(true);
624     })();
625   }
626 });
627
628 const syncInterval = window.setInterval(() => {
629   if (document.visibilityState === 'hidden') {
630     return;
631   }
632
633   void syncUsersFromSupabase();
634 }, USER_SYNC_INTERVAL_MS);
635
636 const handleVisibilityChange = () => {
637   if (document.visibilityState === 'visible') {
638     void syncUsersFromSupabase();
639   }
640 };
641
642 document.addEventListener('visibilitychange', handleVisibilityChange);
643
644 return () => {
645   cancelled = true;

```

```

646     authListener.subscription.unsubscribe();
647     window.clearInterval(syncInterval);
648     document.removeEventListener('visibilitychange', handleVisibilityChange);
649   };
650 }, [resolveAppUserFromAuth, syncUsersFromSupabase]);

```

5. AuthContext.login: ID-to-email lookup and role check

Source: src/context/AuthContext.tsx:652-760

- Validates the selected role, active status, lockout state, and Supabase email binding.
- Signs in with Supabase, verifies the authenticated account matches the app user, then updates login metadata.

Code snippet

```

652   const login = useCallback(
653     async ({ role, id, password, rememberMe = false }: { role: Role; id: string; password: string; rememberMe?: boolean }): Promise<LoginResult> => {
654       const normalizedId = id.trim();
655       const attemptKey = normalizedId || 'role:${role}';
656       const attemptState = attempts[attemptKey];
657       if (attemptState?.lockedUntil && new Date(attemptState.lockedUntil).getTime() > Date.now()) {
658         return { success: false, error: formatRemainingLockout(attemptState.lockedUntil) };
659       }
660
661       if (!normalizedId || !password) {
662         return { success: false, error: 'Enter both your ID and password.' };
663       }
664
665       const registerFailedAttempt = () => {
666         const nextCount = (attemptState?.count ?? 0) + 1;
667         const nextLockedUntil = nextCount >= MAX_ATTEMPTS ? new Date(Date.now() + LOCKOUT_MS).toISOString() : undefined;
668
669         setAttempts((current) => ({
670           ...current,
671           [attemptKey]: {
672             count: nextCount >= MAX_ATTEMPTS ? 0 : nextCount,
673             lockedUntil: nextLockedUntil,
674           },
675         }));
676
677         return nextLockedUntil
678           ? formatRemainingLockout(nextLockedUntil)
679           : 'Invalid credentials. ${MAX_ATTEMPTS - nextCount} attempt${MAX_ATTEMPTS - nextCount === 1 ? '' : 's'} remaining before temporary lockout.';
680       };
681
682       if (!hasSupabaseConfig() && !isLocalDemoModeEnabled()) {
683         return {
684           success: false,
685           error: `${getSupabaseConfigError()} Add the Supabase URL and anon key to the deployment environment and rebuild.',
686         };
687       }
688
689       if (hasSupabaseConfig()) {
690         let matchedUser = usersRef.current.find((account) => account.id.toLowerCase() === normalizedId.toLowerCase());
691         const remoteMatchedUser = await fetchRemoteUserByUniversityId(normalizedId);
692         if (remoteMatchedUser) {
693           matchedUser = remoteMatchedUser;
694         }
695         if (!matchedUser || matchedUser.role !== role) {
696           return { success: false, error: registerFailedAttempt() };

```

```

697     }
698
699     if (matchedUser.status !== 'active') {
700         return { success: false, error: 'This account is inactive. Contact an administrator.' };
701     }
702
703     if (!matchedUser.email) {
704         return { success: false, error: 'This account is missing an email address in public.app_users. Add the email in Supabase and try again.' };
705     }
706
707     const signInResult = await supabaseSignInWithPassword(matchedUser.email, password);
708     if (signInResult.error) {
709         setIsAuthReady(true);
710         return { success: false, error: registerFailedAttempt() };
711     }
712
713     const resolved = await resolveAppUserFromAuth(signInResult.data.user ?? null);
714     if (!resolved || resolved.role !== role || resolved.id.toLowerCase() !== normalizedId.toLowerCase()) {
715         await supabaseSignOut();
716         setIsAuthReady(true);
717         return { success: false, error: 'Your authenticated Supabase account is not linked to the selected app user.' };
718     }
719
720     const lastLogin = new Date().toISOString();
721     setRememberSession(Boolean(rememberMe));
722     setUsers(current) =>
723         current.map((account) =>
724             account.id === resolved.id ? { ...account, lastLogin } : account
725         );
726
727     setUser({ ...resolved });
728     setAttempts(current) => {
729         const next = { ...current };
730         delete next[attemptKey];
731         return next;
732     });
733     setIsAuthReady(true);
734
735     void supabasePatch(
736         'app_users',
737         `university_id=${encodeURIComponent(resolved.id)}`,
738         {
739             last_login_at: lastLogin,
740             status: matchedUser.status,
741             auth_user_id: resolved.authUserId,
742         }
743     ).catch((error) => {
744         console.error('Unable to update Supabase login metadata.', error);
745     });
746
747     void syncUsersFromSupabase();
748     return { success: true };
749 }
750
751 const matchedUser = usersRef.current.find((account) => account.id.toLowerCase() === normalizedId.toLowerCase());
752 if (!matchedUser || matchedUser.role !== role || matchedUser.password !== password) {
753     return { success: false, error: registerFailedAttempt() };
754 }
755
756 if (matchedUser.status !== 'active') {

```

```

757     return { success: false, error: 'This account is inactive. Contact an administrator.' };
758   }
759
760   const nextUser = normalizeManagedUser({ ...matchedUser, lastLogin: new Date().toISOString() });

```

6. Role-based Route Protection

Source: src/App.tsx:25-90

- ProtectedRoute blocks unauthenticated access and redirects users away from routes outside their role.
- The admin transcript-edit route is explicitly admin-only.

Code snippet

```

25  function ProtectedRoute({
26    children,
27    allowedRoles,
28  }: {
29    children: ReactNode;
30    allowedRoles?: Role[];
31  }) {
32    const { isAuthenticated, isAuthReady, user } = useAuth();
33
34    if (!isAuthReady) {
35      return <div className="min-h-screen bg-bg" />;
36    }
37
38    if (!isAuthenticated) {
39      return <Navigate to="/login" replace state={{ authError: 'Your session has expired or you must sign in to continue.' }} />;
40    }
41
42    if (allowedRoles && user && !allowedRoles.includes(user.role)) {
43      return <Navigate to={getHomeRoute(user.role)} replace />;
44    }
45
46    return <>{children}</>;
47  }
48
49  function RoleHomeRedirect() {
50    const { user } = useAuth();
51    return <Navigate to={user ? getHomeRoute(user.role) : '/login'} replace />;
52  }
53
54  export default function App() {
55    return (
56      <BrowserRouter>
57        <AuthProvider>
58          <AppDataProvider>
59            <MessagingProvider>
60              <Routes>
61                <Route path="/" element={<LandingPage />} />
62                <Route path="/login" element={<LoginPage />} />
63                <Route
64                  path="/app"
65                  element={
66                    <ProtectedRoute>
67                      <AppLayout />
68                    </ProtectedRoute>
69                  }
70                >
71                  <Route index element={<RoleHomeRedirect />} />
72                  <Route path="dashboard" element={<ProtectedRoute allowedRoles={['student']}><StudentDashboard /></ProtectedRoute>} />

```

```

73      <Route path="courses" element={<ProtectedRoute allowedRoles={['student']}><CoursePlanner /></ProtectedRoute>} />
74      <Route path="messages" element={<ProtectedRoute allowedRoles={['student']}><StudentMessagesPage /></ProtectedRoute>} />
75      <Route path="profile" element={<Navigate to="/app/dashboard" replace />} />
76      <Route path="settings" element={<ProtectedRoute allowedRoles={['student']}><StudentSettingsPage /></ProtectedRoute>} />
77      <Route path="advisor" element={<ProtectedRoute allowedRoles={['advisor']}><AdvisorDashboard /></ProtectedRoute>} />
78      <Route path="advisor/student/:studentId" element={<ProtectedRoute allowedRoles={['advisor']}><AdvisorStudentDetailPage /></ProtectedRoute>} />
79      <Route path="advisor/messages" element={<ProtectedRoute allowedRoles={['advisor']}><AdvisorMessagesPage /></ProtectedRoute>} />
80      <Route path="advisor/courses" element={<Navigate to="/app/advisor" replace />} />
81      <Route path="advisor/reports" element={<Navigate to="/app/advisor" replace />} />
82      <Route path="advisor/settings" element={<ProtectedRoute allowedRoles={['advisor']}><AdvisorSettingsPage /></ProtectedRoute>} />
83      <Route path="admin" element={<ProtectedRoute allowedRoles={['admin']}><AdminDashboard /></ProtectedRoute>} />
84      <Route path="admin/students" element={<ProtectedRoute allowedRoles={['admin']}><AdminStudentsPage /></ProtectedRoute>} />
85      <Route path="admin/students/:studentId/transcript" element={<ProtectedRoute allowedRoles={['admin']}><AdminStudentTranscriptPage /></ProtectedRoute>} />
86      <Route path="admin/courses" element={<ProtectedRoute allowedRoles={['admin']}><CourseManagement /></ProtectedRoute>} />
87      <Route path="admin/users" element={<ProtectedRoute allowedRoles={['admin']}><UserManagementPage /></ProtectedRoute>} />
88      <Route path="admin/model" element={<Navigate to="/app/admin/courses" replace />} />
89      <Route path="admin/settings" element={<ProtectedRoute allowedRoles={['admin']}><AdminSettingsPage /></ProtectedRoute>} />
90    </Route>

```

7. Remote Data Snapshot: transcript and dashboard views

Source: src/context/AppDataContext.tsx:866-909

- Loads all app data needed after login in one remote snapshot.
- Pulls transcript rows from student_transcript_v and student summaries from student_dashboard_summary_v.

Code snippet

```

866   async function loadRemoteSnapshot(users: ReturnType<typeof useAuth>['users']) {
867     const [
868       academicTermRows,
869       departments,
870       settings,
871       courseRows,
872       prerequisiteRows,
873       corequisiteRows,
874       ruleRows,
875       statRows,
876       profileRows,
877       transcriptViewRows,
878       draftRows,
879       draftCourseRows,
880       evaluationRows,
881       passwordInquiryMessageRows,
882     ] = await Promise.all([
883       supabaseSelect<AcademicTermRow[]>('academic_terms', 'select=term_code,academic_year,term_name,term_type,max_credits'),
884       supabaseSelect<DepartmentRow[]>('departments', 'select=id,name'),
885       supabaseSelect<AppSettingRow[]>('app_settings', 'select=key,value_json'),
886       supabaseSelect<CourseRow[]>(
887         'courses',
888         'select=id,course_code,title,department_id,credits,course_type,is_plannable,internet_difficulty,difficulty_score,difficulty_basis,updated_at&order=course_code.asc
889       ),
890       supabaseSelect<CoursePrerequisiteRow[]>('course_prerequisites', 'select=course_id,prerequisite_course_id'),
891       supabaseSelect<CourseCorequisiteRow[]>('course_corequisites', 'select=course_id,corequisite_course_id'),
892       supabaseSelect<CourseRuleRow[]>('course_rules', 'select=course_id,rule_type,rule_value_int'),
893       supabaseSelect<HistoricalStatRow[]>(
894         'historical_course_stats',
895         'select=id,course_id,term_code,avg_grade,pass_rate,fail_rate,enrollment_count,withdrawals'
896       ),
897       supabaseSelect<StudentProfileRow[]>('student_dashboard_summary_v', 'select=student_id,student_name,department_name,advisor_id,gpa,admission_year,admission_term,completed_credits'),

```

```

898     supabaseSelect<StudentCompletedCourseRow[]>('student_transcript_v', 'select=id,student_id,term_code,term_type,course_code,course_name,credits,final_grade,status,att
empt_no'),
899     supabaseSelect<ScheduleDraftRow[]>('schedule_drafts', 'select=id,student_id,name,term_code,status,saved_at&order=saved_at.desc'),
900     supabaseSelect<ScheduleDraftCourseRow[]>('schedule_draft_courses', 'select=schedule_id,course_id'),
901     supabaseSelect<ScheduleEvaluationRow[]>('
schedule_evaluations',
902     'select=id,schedule_id,student_id,total_score,risk_label,total_credits,model_version,explanation,factors,recommendations,top_courses,evaluated_at&order=evaluated_
at.desc'
903     ),
904     supabaseSelect<PasswordResetInquiryMessageRow[]>(
905     'messages',
906     'select=id,sender_id,body,sent_at,read_at,sender:app_users!messages_sender_id_fkey(university_id,full_name,role)&order=sent_at.desc'
907     ),
908     ],
909 ];

```

8. Mapping Transcript Rows into Frontend State

Source: `src/context/AppDataContext.tsx:1020-1082`

- Converts DB transcript records into `StudentTranscriptRow` objects used by student, advisor, and admin pages.
- Builds completed-course sets and student profile summaries from transcript state.

Code snippet

```

1020 const completedCourseCodesByStudentId = new Map<string, Set<string>>();
1021 const transcriptRows: StudentTranscriptRow[] = [];
1022 transcriptViewRows.forEach((row) => {
1023     const studentUniversityId = appUsersByAppId.get(row.student_id)?.id;
1024     if (!studentUniversityId) {
1025         return;
1026     }
1027
1028     if (row.status === 'passed') {
1029         const existingCompleted = completedCourseCodesByStudentId.get(studentUniversityId) ?? new Set<string>();
1030         existingCompleted.add(row.course_code);
1031         completedCourseCodesByStudentId.set(studentUniversityId, existingCompleted);
1032     }
1033
1034     const termCode = normalizeTermCode(row.term_code);
1035     transcriptRows.push({
1036         id: row.id,
1037         studentId: studentUniversityId,
1038         termCode,
1039         termLabel: formatTermLabel(termCode),
1040         termType: row.term_type,
1041         courseCode: row.course_code,
1042         courseName: row.course_name,
1043         credits: row.credits,
1044         finalGrade: row.final_grade === null ? null : Number(row.final_grade),
1045         status: row.status,
1046         attemptNo: row.attempt_no,
1047     });
1048 });
1049 transcriptRows.sort((left, right) => {
1050     const termCompare = compareTermCodesNewestFirst(left.termCode, right.termCode);
1051     if (termCompare !== 0) {
1052         return termCompare;
1053     }
1054
1055     const attemptCompare = (right.attemptNo ?? 1) - (left.attemptNo ?? 1);
1056     if (attemptCompare !== 0) {
1057         return attemptCompare;

```

```

1058     }
1059
1060     return left.courseCode.localeCompare(right.courseCode);
1061   });
1062
1063   const remoteProfiles = profileRows.flatMap((row) => {
1064     const student = appUsersByAppId.get(row.student_id);
1065     if (!student) {
1066       return [];
1067     }
1068
1069     const advisor = row.advisor_id ? appUsersByAppId.get(row.advisor_id) : null;
1070
1071     return [{
1072       id: student.id,
1073       name: row.student_name,
1074       gpa: Number(row.gpa ?? 0),
1075       creditsCompleted: row.completed_credits,
1076       department: row.department_name,
1077       advisorId: advisor?.id ?? '',
1078       completedCourseCodes: [...(completedCourseCodesByStudentId.get(student.id) ?? new Set<string>())],
1079       admissionYear: row.admission_year ?? (Number(student.id.slice(0, 4)) || new Date().getFullYear()),
1080       admissionTerm: row.admission_term ?? 'fall',
1081     } satisfies StudentProfile];
1082   });

```

9. Shared Transcript Accessor

Source: `src/context/AppDataContext.tsx:1264-1348`

- Builds the transcript view consumed by dashboards.
- Merges recorded marks with catalog courses so untaken courses still appear in the transcript table.

Code snippet

```

1264   const studentInsights = useMemo(
1265     () => buildStudentInsights(state.studentProfiles, state.scheduleDrafts.filter(isUserVisibleScheduleDraft)),
1266     [state.scheduleDrafts, state.studentProfiles]
1267   );
1268
1269   const getStudentProfile = useCallback(
1270     (studentId: string) =>
1271       state.studentProfiles.find((profile) => profile.id === studentId)
1272       ?? null,
1273     [state.studentProfiles]
1274   );
1275
1276   const getStudentTranscript = useCallback(
1277     (studentId: string) => {
1278       const profile = getStudentProfile(studentId);
1279       const recordedRows = state.transcriptRows.filter((row) => row.studentId === studentId);
1280
1281       if (!profile) {
1282         return recordedRows;
1283       }
1284
1285       const latestRecordedByCode = new Map<string, StudentTranscriptRow>();
1286       recordedRows.forEach((row) => {
1287         if (!latestRecordedByCode.has(row.courseCode)) {
1288           latestRecordedByCode.set(row.courseCode, row);
1289         }
1290       });

```

```

1291
1292     const transcriptRows = state.courses.map((course) => {
1293         const recorded = latestRecordedByCode.get(course.code);
1294         if (recorded) {
1295             return {
1296                 ...recorded,
1297                 termLabel: formatCompactTermLabel(recorded.termCode),
1298             } satisfies StudentTranscriptRow;
1299         }
1300
1301         return {
1302             studentId,
1303             termCode: '',
1304             termLabel: '-',
1305             termType: 'regular',
1306             courseCode: course.code,
1307             courseName: course.name,
1308             credits: course.credits,
1309             finalGrade: null,
1310             status: 'not_taken',
1311             attemptNo: 0,
1312         } satisfies StudentTranscriptRow;
1313     });
1314
1315     return transcriptRows.sort((left, right) => {
1316         const leftTaken = left.termCode ? 1 : 0;
1317         const rightTaken = right.termCode ? 1 : 0;
1318         if (leftTaken !== rightTaken) {
1319             return rightTaken - leftTaken;
1320         }
1321
1322         if (left.termCode && right.termCode) {
1323             const termCompare = compareTermCodesOldestFirst(left.termCode, right.termCode);
1324             if (termCompare !== 0) {
1325                 return termCompare;
1326             }
1327         }
1328
1329         return left.courseCode.localeCompare(right.courseCode);
1330     });
1331 },
1332 [getStudentProfile, state.courses, state.transcriptRows]
1333 );
1334
1335 const getStudentTermMetrics = useCallback(
1336     (studentId: string) =>
1337         state.termMetrics
1338             .filter((row) => row.studentId === studentId)
1339             .sort((left, right) => compareTermCodesOldestFirst(left.termCode, right.termCode)),
1340     [state.termMetrics]
1341 );
1342
1343 const getStudentTranscriptSemesters = useCallback(
1344     (studentId: string) => buildTranscriptSemesters(studentId, state.transcriptRows, state.termMetrics),
1345     [state.termMetrics, state.transcriptRows]
1346 );
1347
1348 const getStudentAvailableTerms = useCallback(

```

10. Advisor Student Detail: ownership guard

Source: src/pages/advisor/AdvisorStudentDetailPage.tsx:20-110

- Advisor page only resolves a profile when the student belongs to the logged-in advisor.
- Advisors can inspect schedule drafts, risk, GPA, and transcript rows.

Code snippet

```
20 export default function AdvisorStudentDetailPage() {
21   const { studentId = '' } = useParams();
22   const navigate = useNavigate();
23   const { user } = useAuth();
24   const {
25     courses,
26     getSelectedCourses,
27     getStudentDrafts,
28     getStudentTranscriptSemesters,
29     getStudentTermMetrics,
30     getStudentTranscript,
31     isAppDataReady,
32     plannerReviewSnapshots,
33     studentInsights,
34   } = useAppData();
35
36   const profile = studentInsights.find((item) => item.id === studentId && item.advisorId === user?.id) ?? null;
37   const selectedCourses = getSelectedCourses(studentId);
38   const drafts = getStudentDrafts(studentId);
39   const transcriptRows = getStudentTranscript(studentId);
40   const transcriptSemesters = getStudentTranscriptSemesters(studentId);
41   const termMetrics = getStudentTermMetrics(studentId);
42   const currentEvaluation = plannerReviewSnapshots[studentId] ?? null;
43   const [activeTab, setActiveTab] = useState<'overview' | 'transcript' | 'semester-transcript'>('overview');
44   const [selectedSemesterTermCode, setSelectedSemesterTermCode] = useState('');
45   const [isDraftsOpen, setIsDraftsOpen] = useState(false);
46   const [selectedDraftId, setSelectedDraftId] = useState<string | null>(null);
47
48   const selectedCoursesForDisplay = useMemo(
49     () =>
50       selectedCourses.length > 0
51         ? selectedCourses
52         : drafts[0]
53           ? drafts[0].courseCodes
54             .map((code) => courses.find((course) => course.code === code))
55             .filter((course): course is NonNullable<typeof course> => Boolean(course))
56           : [],
57     [courses, drafts, selectedCourses]
58   );
59
60   const totalCredits = useMemo(
61     () => selectedCoursesForDisplay.reduce((sum, course) => sum + course.credits, 0),
62     [selectedCoursesForDisplay]
63   );
64
65   const score = currentEvaluation?.totalScore ?? drafts[0]?.evaluation.totalScore ?? null;
66   const diffInfo = score !== null ? getDiffLabel(score) : null;
67   const meterPct = score !== null ? clamp(score, 0, 100) : 50;
68   const explanation = currentEvaluation?.explanation ?? drafts[0]?.evaluation.explanation ?? [];
69   const selectedDraft = drafts.find((draft) => draft.id === selectedDraftId) ?? null;
70   const selectedDraftCourses = useMemo(
71     () =>
72       selectedDraft
```

```

73     ? selectedDraft.courseCodes
74     .map((code) => courses.find((course) => course.code === code))
75     .filter((course): course is NonNullable<typeof course> => Boolean(course))
76     : [],
77     [courses, selectedDraft]
78 );
79 const selectedSemester = transcriptSemesters.find((semester) => semester.termCode === selectedSemesterTermCode)
80 ?? transcriptSemesters[0]
81 ?? null;
82
83 const handleMessageStudent = () => {
84     if (!profile) {
85         return;
86     }
87
88     navigate('/app/advisor/messages', { state: { focusUserId: profile.id, scrollToBottom: true } });
89 };
90
91 const handleOpenDrafts = () => {
92     setSelectedDraftId(null);
93     setIsDraftsOpen(true);
94 };
95
96 const handleCloseDrafts = () => {
97     setSelectedDraftId(null);
98     setIsDraftsOpen(false);
99 };
100
101 if (!isAppDataReady) {
102     return (
103         <div className="rounded-xl border border-gray-200 bg-white p-8 text-sm text-gray-500">
104             Loading student details...
105         </div>
106     );
107 }
108
109 if (!profile) {
110     return (

```

11. Advisor Transcript Display

Source: src/pages/advisor/AdvisorStudentDetailPage.tsx:380-488

- Shows advisor-facing full transcript and semester transcript views.
- Important boundary: this page displays marks, but does not mutate them.

Code snippet

```

380     <div className="mb-4">
381         <h2 className="text-lg font-bold text-[#0f1e3c]">Full Transcript</h2>
382         <p className="mt-1 text-sm text-gray-500">All degree courses for this advisee, including untaken courses and recorded marks.</p>
383     </div>
384     {transcriptRows.length > 0 ? (
385         <div className="overflow-hidden rounded-xl border border-gray-200">
386             <div className="flex items-center justify-between bg-slate-50 px-4 py-3">
387                 <p className="font-semibold text-[#0f1e3c]">Degree Transcript</p>
388                 <span className="text-xs font-medium text-gray-500">{transcriptRows.length} course{transcriptRows.length !== 1 ? 's' : ''}</span>
389             </div>
390             <div className="overflow-x-auto">
391                 <table className="w-full text-sm">
392                     <thead>
393                         <tr className="border-b border-gray-100 text-left text-xs uppercase tracking-wider text-gray-400">

```

```

394         <th className="px-4 py-3 pr-4">Code</th>
395         <th className="px-4 py-3 pr-4">Course</th>
396         <th className="px-4 py-3 pr-4 text-center">Credits</th>
397         <th className="px-4 py-3 pr-4 text-center">Term</th>
398         <th className="px-4 py-3 text-center">Grade</th>
399     </tr>
400 </thead>
401 <tbody>
402     {transcriptRows.map((row) => (
403         <tr key={row.courseCode} className="border-b border-gray-50 last:border-0">
404             <td className="px-4 py-3 font-mono font-semibold text-[#0f1e3c]">{row.courseCode}</td>
405             <td className="px-4 py-3 text-gray-700">{row.courseName}</td>
406             <td className="px-4 py-3 text-center text-gray-600">{row.credits}</td>
407             <td className="px-4 py-3 text-center text-gray-600">{row.termLabel}</td>
408             <td className="px-4 py-3 text-center font-semibold text-[#0f1e3c]">{row.finalGrade === null ? '-' : row.finalGrade.toFixed(0)}</td>
409         </tr>
410     )})
411 </tbody>
412 </table>
413 </div>
414 </div>
415 ) : (
416     <div className="rounded-lg border border-dashed border-gray-300 p-6 text-sm text-gray-500">
417         No transcript rows are available yet.
418     </div>
419 )}
420 </div>
421 ) : (
422     <div className="rounded-xl border border-gray-200 bg-white p-4 sm:p-6">
423         <div className="mb-4 flex flex-col gap-4 sm:flex-row sm:items-end sm:justify-between">
424             <div>
425                 <h2 className="text-lg font-bold text-[#0f1e3c]">Semester Transcript</h2>
426                 <p className="mt-1 text-sm text-gray-500">Review the selected student's recorded courses and marks semester by semester.</p>
427             </div>
428             <label className="flex flex-col gap-1 text-sm font-medium text-gray-600">
429                 Semester
430                 <select
431                     value={selectedSemester?.termCode ?? ''}
432                     onChange={(event) => setSelectedSemesterTermCode(event.target.value)}
433                     className="min-w-[200px] rounded-lg border border-gray-200 bg-white px-3 py-2 text-sm text-[#0f1e3c] outline-none ring-0 transition-colors focus:border-
[#2563eb]"
434                 >
435                     {transcriptSemesters.map((semester) => (
436                         <option key={semester.termCode} value={semester.termCode}>
437                             {semester.termLabel}
438                         </option>
439                     )})}
440                 </select>
441             </label>
442         </div>
443         {selectedSemester ? (
444             <div className="overflow-hidden rounded-xl border border-gray-200">
445                 <div className="flex flex-wrap items-center justify-between gap-3 bg-slate-50 px-4 py-3">
446                     <div>
447                         <p className="font-semibold text-[#0f1e3c]">{selectedSemester.termLabel}</p>
448                         <p className="text-xs text-gray-500">
449                             {selectedSemester.completedCredits} hour{selectedSemester.completedCredits !== 1 ? 's' : ''} | {selectedSemester.courseCount} course{selectedSemeste
r.courseCount !== 1 ? 's' : ''} | GPA {selectedSemester.gpa?.toFixed(2) ?? '-'}
450                     </p>
451                 </div>

```

```

452     </div>
453     <div className="overflow-x-auto">
454       <table className="w-full text-sm">
455         <thead>
456           <tr className="border-b border-gray-100 text-left text-xs uppercase tracking-wider text-gray-400">
457             <th className="px-4 py-3 pr-4">Code</th>
458             <th className="px-4 py-3 pr-4">Course</th>
459             <th className="px-4 py-3 pr-4 text-center">Credits</th>
460             <th className="px-4 py-3 pr-4 text-center">Status</th>
461             <th className="px-4 py-3 text-center">Grade</th>
462           </tr>
463         </thead>
464         <tbody>
465           {selectedSemester.rows.map((row) => (
466             <tr key={` ${row.termCode} - ${row.courseCode}`} className="border-b border-gray-50 last:border-0">
467               <td className="px-4 py-3 font-mono font-semibold text-[#0f1e3c]">{row.courseCode}</td>
468               <td className="px-4 py-3 text-gray-700">{row.courseName}</td>
469               <td className="px-4 py-3 text-center text-gray-600">{row.credits}</td>
470               <td className="px-4 py-3 text-center">
471                 <span className="rounded-full bg-emerald-50 px-2 py-0.5 text-[10px] font-semibold uppercase tracking-wide text-emerald-700">
472                   {row.status.replace('_', ' ')}
473                 </span>
474               </td>
475               <td className="px-4 py-3 text-center font-semibold text-[#0f1e3c]">{row.finalGrade === null ? '-' : row.finalGrade.toFixed(0)}</td>
476             </tr>
477           )})
478         </tbody>
479       </table>
480     </div>
481   </div>
482 ) : (
483   <div className="rounded-lg border border-dashed border-gray-300 p-6 text-sm text-gray-500">
484     No semester transcript data is available yet.
485   </div>
486 )}
487 </div>
488 )}

```

12. Admin Student List to Transcript Editor

Source: `src/pages/admin/AdminStudentsPage.tsx:7-111`

- Filters users to students and links admins into `/app/admin/students/:studentId/transcript`.
- This is the UI entry point for changing a mark.

Code snippet

```

7  export default function AdminStudentsPage() {
8    const { users } = useAuth();
9    const {
10      isAppDataReady,
11      studentInsights,
12    } = useAppData();
13    const [search, setSearch] = useState('');
14    const deferredSearch = useDeferredValue(search);
15
16    const students = useMemo(
17      () => users.filter((account) => account.role === 'student'),
18      [users]
19    );
20    const insightById = useMemo(
21      () => new Map(studentInsights.map((student) => [student.id, student])),

```

```

22     [studentInsights]
23   );
24
25   const filteredStudents = useMemo(() => {
26     const query = deferredSearch.trim().toLowerCase();
27     if (!query) {
28       return students;
29     }
30
31     return students.filter((student) => (
32       student.name.toLowerCase().includes(query) ||
33       student.id.toLowerCase().includes(query)
34     ));
35   }, [deferredSearch, students]);
36
37   return (
38     <section className="rounded-xl border border-gray-200 bg-white p-4 sm:p-6">
39       <div className="mb-4 flex flex-wrap items-center justify-between gap-3">
40         <h2 className="flex items-center gap-2 text-lg font-bold text-[#0f1e3c]">
41           <GraduationCap className="h-5 w-5 text-[#2563eb]" />
42           Students
43         </h2>
44         <div className="relative w-full sm:max-w-sm">
45           <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-gray-400" />
46           <input
47             type="text"
48             value={search}
49             onChange={(event) => setSearch(event.target.value)}
50             placeholder="Search by student name or ID"
51             className="w-full rounded-lg border border-gray-200 py-2 pl-10 pr-4 text-sm focus:border-[#2563eb] focus:outline-none focus:ring-2 focus:ring-[#2563eb]/30"
52           />
53         </div>
54       </div>
55
56       {!isAppDataReady ? (
57         <div className="rounded-lg border border-dashed border-gray-200 p-6 text-sm text-gray-500">
58           Loading students...
59         </div>
60       ) : (
61         <div className="overflow-x-auto">
62           <table className="w-full text-sm">
63             <thead>
64               <tr className="border-b border-gray-100 text-left text-xs uppercase tracking-wider text-gray-400">
65                 <th className="pb-2 pr-4">Student</th>
66                 <th className="pb-2 pr-4">Advisor</th>
67                 <th className="pb-2 pr-4 text-center">GPA</th>
68                 <th className="pb-2 pr-4 text-center">Credits</th>
69                 <th className="pb-2 text-right">Action</th>
70               </tr>
71             </thead>
72             <tbody>
73               {filteredStudents.map((student) => {
74                 const insight = insightById.get(student.id);
75                 const advisor = insight?.advisorId ? users.find((account) => account.id === insight.advisorId) : null;
76                 return (
77                   <tr key={student.id} className="border-b border-gray-50 last:border-0 hover:bg-gray-50/60">
78                     <td className="py-2.5 pr-4">
79                       <p className="font-semibold text-[#0f1e3c]">{student.name}</p>
80                       <p className="text-xs text-gray-400">{student.id}</p>
81                     </td>

```

```

82         <td className="py-2.5 pr-4 text-gray-600">{advisor?.name ?? 'Unassigned'}</td>
83         <td className="py-2.5 pr-4 text-center font-semibold text-[#0f1e3c]">{insight ? insight.gpa.toFixed(2) : '-'}</td>
84         <td className="py-2.5 pr-4 text-center text-gray-600">{insight?.creditsCompleted ?? '-'}</td>
85         <td className="py-2.5 text-right">
86             <div className="flex justify-end gap-2">
87                 <Link
88                     to={'/app/admin/students/${student.id}/transcript'}
89                     className="inline-flex items-center justify-center rounded-lg border border-[#2563eb]/20 bg-[#2563eb]/5 px-3 py-1.5 text-xs font-semibold text
-#[2563eb] transition-colors hover:bg-[#2563eb]/10"
90                 >
91                     View transcript
92                 </Link>
93             </div>
94         </td>
95     </tr>
96 };
97 }}}
98 {filteredStudents.length === 0 && (
99     <tr>
100         <td colSpan={5} className="py-8 text-center text-sm text-gray-500">
101             No students match this search.
102         </td>
103     </tr>
104 }}
105 </tbody>
106 </table>
107 </div>
108 )}
109 </section>
110 );
111 }

```

13. Admin Transcript Editor: validation and save loop

Source: src/pages/admin/AdminStudentTranscriptPage.tsx:85-223

- Tracks edited draft rows and validates whole-number marks from 35 to 99 plus attempt numbers from 1 to 10.
- Only changed rows are saved; each row is sent through upsertTranscriptEntry.

Code snippet

```

85 export default function AdminStudentTranscriptPage() {
86     const { studentId } = useParams();
87     const { users } = useAuth();
88     const {
89         getStudentAvailableTerms,
90         getStudentTranscript,
91         isAppDataReady,
92         studentInsights,
93         upsertTranscriptEntry,
94     } = useAppData();
95     const [draftRows, setDraftRows] = useState<Record<string, TranscriptDraftRow>>({});
96     const [message, setMessage] = useState<string | null>(null);
97     const [error, setError] = useState<string | null>(null);
98     const [isSaving, setIsSaving] = useState(false);
99
100     const student = users.find((account) => account.id === studentId && account.role === 'student');
101     const advisor = getAdvisor(student, users, studentInsights);
102     const transcriptRows = useMemo(
103         () => studentId ? getStudentTranscript(studentId) : [],
104         [getStudentTranscript, studentId]
105     );

```

```

106 const termOptions = useMemo(() => {
107   if (!studentId) {
108     return [];
109   }
110
111   const terms = new Map<string, { termCode: string; termType: ReturnType<typeof getTermTypeFromCode> }>();
112   getStudentAvailableTerms(studentId).forEach((term) => terms.set(term.termCode, term));
113   transcriptRows.forEach((row) => terms.set(row.termCode, { termCode: row.termCode, termType: row.termType }));
114   return [...terms.values()].map((term) => ({
115     ...term,
116     termLabel: formatTermLabel(term.termCode),
117   }));
118 }, [getStudentAvailableTerms, studentId, transcriptRows]);
119
120 if (!studentId || (!student && isAppDataReady)) {
121   return <Navigate to="/app/admin/students" replace />;
122 }
123
124 const getDraftForRow = (row: StudentTranscriptRow) => {
125   const key = getDraftKey(row);
126   return draftRows[key] ?? {
127     finalGrade: formatGradeValue(row.finalGrade),
128     attemptNo: formatAttemptValue(row.attemptNo),
129     termCode: row.termCode || termOptions[0]?.termCode || '',
130   };
131 };
132
133 const validationErrors = transcriptRows.flatMap((row) => {
134   const draft = getDraftForRow(row);
135   const finalGrade = parseGradeValue(draft.finalGrade);
136   const attemptNo = parseAttemptValue(draft.attemptNo);
137   if (draft.finalGrade.trim()) {
138     if (!/^\\d+$/\\.test(draft.finalGrade.trim())) {
139       return ['${row.courseCode} needs a whole-number mark, or a blank mark.'];
140     }
141
142     if (!Number.isInteger(finalGrade) || finalGrade === null || finalGrade < 35 || finalGrade > 99) {
143       return ['${row.courseCode} needs a whole-number mark from 35 to 99, or a blank mark.'];
144     }
145
146     if (!draft.attemptNo.trim()) {
147       return ['${row.courseCode} needs an attempt number from 1 to 10 when a mark is entered.'];
148     }
149   }
150
151   if (draft.attemptNo.trim()) {
152     if (!/^\\d+$/\\.test(draft.attemptNo.trim())) {
153       return ['${row.courseCode} needs a whole-number attempt from 1 to 10, or a blank attempt.'];
154     }
155
156     if (!Number.isInteger(attemptNo) || attemptNo === null || attemptNo < 1 || attemptNo > 10) {
157       return ['${row.courseCode} needs a whole-number attempt from 1 to 10, or a blank attempt.'];
158     }
159   }
160
161   return [];
162 });
163
164 const changedRows = transcriptRows.filter((row) => {
165   const draft = draftRows[getDraftKey(row)];

```

```

166     if (!draft) {
167         return false;
168     }
169
170     return !row.id
171         || draft.termCode !== row.termCode
172         || parseGradeValue(draft.finalGrade) !== row.finalGrade
173         || parseAttemptValue(draft.attemptNo) !== row.attemptNo;
174 });
175 const hasChanges = changedRows.length > 0;
176
177 const transcriptByTerm = transcriptRows.reduce<Record<string, StudentTranscriptRow[]>>((groups, row) => {
178     const draft = getDraftForRow(row);
179     const termCode = draft.termCode;
180     groups[termCode] = [...(groups[termCode] ?? []), row];
181     return groups;
182 }, {});
183
184 const handleReset = () => {
185     setDraftRows({});
186     setMessage(null);
187     setError(null);
188 };
189
190 const handleSave = async () => {
191     setMessage(null);
192     setError(null);
193
194     if (validationErrors.length > 0) {
195         setError(validationErrors[0]);
196         return;
197     }
198
199     setIsSaving(true);
200     for (const row of changedRows) {
201         const draft = draftRows[getDraftKey(row)];
202         const finalGrade = parseGradeValue(draft.finalGrade);
203         const attemptNo = parseAttemptValue(draft.attemptNo);
204         const result = await upsertTranscriptEntry({
205             id: row.id,
206             studentId: row.studentId,
207             termCode: draft.termCode || row.termCode || termOptions[0]?.termCode || '',
208             courseCode: row.courseCode,
209             finalGrade,
210             status: getStatusFromGrade(finalGrade),
211             attemptNo: attemptNo ?? Math.max(row.attemptNo, 1),
212         });
213
214         if (!result.success) {
215             setIsSaving(false);
216             setError(result.error ?? 'Unable to save ${row.courseCode}.');
217             return;
218         }
219     }
220     setIsSaving(false);
221     setDraftRows({});
222     setMessage('Transcript changes saved.');
```

14. Admin Transcript Editor: mark inputs

Source: src/pages/admin/AdminStudentTranscriptPage.tsx:265-373

- Shows the exact inputs used to change attempt number, semester taken, mark, and derived status.
- The Save button above this table persists draftRows through handleSave.

Code snippet

```
265     <section className="rounded-xl border border-gray-200 bg-white p-4 sm:p-6">
266       {transcriptRows.length > 0 ? (
267         <div className="space-y-5">
268           {Object.entries(transcriptByTerm).map(([termCode, rows]) => (
269             <div key={termCode} className="overflow-hidden rounded-xl border border-gray-200">
270               <div className="flex items-center justify-between bg-slate-50 px-4 py-3">
271                 <p className="font-semibold text-[#0f1e3c]">{formatTermLabel(termCode)}</p>
272                 <span className="text-xs font-medium text-gray-500">{rows.length} course{rows.length !== 1 ? 's' : ''}</span>
273               </div>
274               <div className="overflow-x-auto">
275                 <table className="w-full text-sm">
276                   <thead>
277                     <tr className="border-b border-gray-100 text-left text-xs uppercase tracking-wider text-gray-400">
278                       <th className="px-4 py-3 pr-4">Code</th>
279                       <th className="px-4 py-3 pr-4">Course</th>
280                       <th className="px-4 py-3 pr-4 text-center">Credits</th>
281                       <th className="px-4 py-3 pr-4 text-center">Attempt</th>
282                       <th className="px-4 py-3 pr-4">Semester taken</th>
283                       <th className="px-4 py-3 pr-4">Mark</th>
284                       <th className="px-4 py-3 text-center">Status</th>
285                     </tr>
286                   </thead>
287                   <tbody>
288                     {rows.map((row) => {
289                       const key = getDraftKey(row);
290                       const draft = getDraftForRow(row);
291                       const parsedGrade = parseGradeValue(draft.finalGrade);
292                       return (
293                         <tr key={key} className="border-b border-gray-50 last:border-0">
294                           <td className="px-4 py-3 font-mono font-semibold text-[#0f1e3c]">{row.courseCode}</td>
295                           <td className="min-w-56 px-4 py-3 text-gray-700">{row.courseName}</td>
296                           <td className="px-4 py-3 text-center text-gray-600">{row.credits}</td>
297                           <td className="px-4 py-3 text-center">
298                             <input
299                               type="number"
300                               min="1"
301                               max="10"
302                               step="1"
303                               inputMode="numeric"
304                               value={draft.attemptNo}
305                               onChange={(event) => setDraftRows((current) => ({
306                                 ...current,
307                                 [key]: { ...draft, attemptNo: event.target.value },
308                               })))>
309                             <div>
310                               <div>
311                                 <div>
312                                   <td className="px-4 py-3">
313                                     <select
314                                       value={draft.termCode}
315                                       onChange={(event) => setDraftRows((current) => ({
316                                         ...current,
```

```

317         [key]: { ...draft, termCode: event.target.value },
318     })))
319     className="w-44 rounded-lg border border-gray-200 px-3 py-2 text-sm focus:border-[#2563eb] focus:outline-none focus:ring-2 focus:ring-[#
2563eb]/30"
320     >
321     {termOptions.map((term) => (
322         <option key={term.termCode} value={term.termCode}>{term.termLabel}</option>
323     ))}
324     </select>
325 </td>
326 <td className="px-4 py-3">
327     <input
328         type="number"
329         min="35"
330         max="99"
331         step="1"
332         inputMode="numeric"
333         value={draft.finalGrade}
334         onChange={(event) => setDraftRows((current) => ({
335             ...current,
336             [key]: { ...draft, finalGrade: event.target.value },
337         })))
338         className="w-28 rounded-lg border border-gray-200 px-3 py-2 text-sm focus:border-[#2563eb] focus:outline-none focus:ring-2 focus:ring-[#
2563eb]/30"
339     />
340     <button
341         type="button"
342         onClick={() => setDraftRows((current) => ({
343             ...current,
344             [key]: { ...draft, finalGrade: '' },
345         })))
346         className="mt-2 text-xs font-semibold text-gray-500 transition-colors hover:text-[#2563eb]"
347     >
348         Clear grade
349     </button>
350 </td>
351 <td className="px-4 py-3 text-center">
352     <span className={'inline-flex rounded-full px-2.5 py-0.5 text-[10px] font-semibold ${getStatusClass(parsedGrade)}'}>
353         {getStatusLabel(parsedGrade)}
354     </span>
355 </td>
356 </tr>
357 );
358 }])
359 </tbody>
360 </table>
361 </div>
362 </div>
363 )])
364 </div>
365 ) : (
366     <div className="rounded-lg border border-dashed border-gray-300 p-6 text-sm text-gray-500">
367         No transcript rows are available for this student yet.
368     </div>
369 )}
370 </section>
371 </div>
372 );
373 }

```

15. AppData upsertTranscriptEntry: client validation, API call, local sync

Source: src/context/AppDataContext.tsx:1980-2054

- Normalizes status from the grade, revalidates against catalog and current transcript rows, then calls the admin API.
- After success, syncDerivedTranscriptState updates GPA, completed credits, and transcript state in the UI.

Code snippet

```
1980     const termLimit = getCreditLimitForTermCode(input.termCode);
1981     if (nextTermHours > termLimit) {
1982       return `${formatTermLabel(input.termCode)} cannot exceed ${termLimit} hours.`;
1983     }
1984
1985     const duplicatePassed = currentRows.some((row) =>
1986       row.studentId === input.studentId
1987       && row.courseCode === input.courseCode
1988       && row.status === 'passed'
1989       && row.id !== input.id
1990     );
1991     if (input.status === 'passed' && duplicatePassed && input.attemptNo <= 1) {
1992       return 'This course was already passed. Use attempt number 2 or higher for an intentional retake.';
1993     }
1994
1995     return null;
1996   }, [state.courses]);
1997
1998   const upsertTranscriptEntry = useCallback(async (input: TranscriptEntryInput): Promise<PlannerActionResult> => {
1999     const normalizedStatus = getTranscriptStatusForGrade(input.finalGrade, input.status);
2000     const normalizedInput = { ...input, status: normalizedStatus } satisfies TranscriptEntryInput;
2001     const validationError = validateTranscriptInput(normalizedInput, state.transcriptRows);
2002     if (validationError) {
2003       return { success: false, error: validationError };
2004     }
2005
2006     const course = state.courses.find((item) => item.code === normalizedInput.courseCode);
2007     if (!course) {
2008       return { success: false, error: 'Course was not found.' };
2009     }
2010
2011     let entryId = normalizedInput.id ?? createId();
2012
2013     if (hasSupabaseConfig()) {
2014       try {
2015         const result = await callAdminDataEndpoint<{ id?: string }>('/api/admin-upsert-transcript-entry', {
2016           id: entryId,
2017           existingEntry: Boolean(normalizedInput.id),
2018           studentId: normalizedInput.studentId,
2019           termCode: normalizedInput.termCode,
2020           courseCode: normalizedInput.courseCode,
2021           finalGrade: normalizedInput.finalGrade,
2022           status: normalizedStatus,
2023           attemptNo: normalizedInput.attemptNo,
2024         });
2025         entryId = result?.id ?? entryId;
2026       } catch (error) {
2027         return {
2028           success: false,
2029           error: error instanceof Error ? error.message : 'Unable to save transcript entry.',
2030         };

```

```

2031     }
2032   }
2033
2034   const nextRow = {
2035     id: entryId,
2036     studentId: normalizedInput.studentId,
2037     termCode: normalizedInput.termCode,
2038     termLabel: formatTermLabel(normalizedInput.termCode),
2039     termType: /summer/i.test(normalizedInput.termCode) ? 'summer' : 'regular',
2040     courseCode: course.code,
2041     courseName: course.name,
2042     credits: course.credits,
2043     finalGrade: normalizedInput.finalGrade,
2044     status: normalizedStatus,
2045     attemptNo: normalizedInput.attemptNo,
2046   } satisfies StudentTranscriptRow;
2047
2048   setState((current) => syncDerivedTranscriptState(current, [
2049     ...current.transcriptRows.filter((row) => row.id !== entryId),
2050     nextRow,
2051   ]));
2052
2053   return { success: true };
2054 }, [state.courses, state.transcriptRows, validateTranscriptInput]);

```

16. Admin Transcript API: service-role write with admin guard

Source: `api/admin-upsert-transcript-entry.ts:17-201`

- Uses the Supabase service-role client only on the server.
- Verifies the bearer token belongs to an admin app user before updating or upserting `student_transcript_entries`.

Code snippet

```

17 function getSupabaseAdminClient() {
18   const supabaseUrl = process.env.SUPABASE_URL ?? process.env.VITE_SUPABASE_URL;
19   const serviceRoleKey = process.env.SUPABASE_SERVICE_ROLE_KEY;
20
21   if (!supabaseUrl || !serviceRoleKey) {
22     throw new Error('Supabase admin environment is not configured.');
```

```

43 }
44
45 async function requireAdmin(request: VercelRequest, supabase: ReturnType<typeof getSupabaseAdminClient>) {
46   const token = String(request.headers.authorization ?? '').replace(/^Bearer\s+/i, '').trim();
47   if (!token) {
48     throw new Error('Missing admin session token.');
```

```

103     existingEntry: payload.existingEntry === true,
104     studentId,
105     termCode,
106     courseCode,
107     finalGrade,
108     status: getStatusForGrade(finalGrade),
109     attemptNo,
110   },
111 };
112 }
113
114 export default async function handler(request: VercelRequest, response: VercelResponse) {
115   if (request.method !== 'POST') {
116     response.status(405).json({ success: false, error: 'Method not allowed.' });
117     return;
118   }
119
120   try {
121     const payload = typeof request.body === 'string'
122       ? JSON.parse(request.body) as TranscriptEntryPayload
123       : request.body as TranscriptEntryPayload;
124     const validation = validatePayload(payload);
125     if ('error' in validation) {
126       response.status(400).json({ success: false, error: validation.error });
127       return;
128     }
129
130     const supabase = getSupabaseAdminClient();
131     await requireAdmin(request, supabase);
132
133     const { input } = validation;
134     const { data: student, error: studentError } = await supabase
135       .from('app_users')
136       .select('id')
137       .eq('university_id', input.studentId)
138       .eq('role', 'student')
139       .maybeSingle();
140
141     if (studentError) {
142       throw studentError;
143     }
144
145     if (!student) {
146       response.status(404).json({ success: false, error: 'Student account was not found.' });
147       return;
148     }
149
150     const { data: course, error: courseError } = await supabase
151       .from('courses')
152       .select('id')
153       .eq('course_code', input.courseCode)
154       .maybeSingle();
155
156     if (courseError) {
157       throw courseError;
158     }
159
160     if (!course) {
161       response.status(404).json({ success: false, error: 'Course was not found.' });
162       return;

```

```

163     }
164
165     const transcriptPayload = {
166       id: input.id,
167       student_id: student.id,
168       term_code: input.termCode,
169       course_id: course.id,
170       final_grade: input.finalGrade,
171       status: input.status,
172       attempt_no: input.attemptNo,
173     };
174
175     const query = input.existingEntry && input.id
176       ? supabase
177         .from('student_transcript_entries')
178         .update(transcriptPayload)
179         .eq('id', input.id)
180         .select('id')
181         .single()
182       : supabase
183         .from('student_transcript_entries')
184         .upsert(transcriptPayload, { onConflict: 'student_id,course_id,term_code,attempt_no' })
185         .select('id')
186         .single();
187
188     const { data: savedEntry, error: saveError } = await query;
189     if (saveError) {
190       throw saveError;
191     }
192
193     response.status(200).json({ success: true, id: savedEntry.id });
194   } catch (error) {
195     console.error('Admin transcript upsert failed.', error);
196     response.status(500).json({
197       success: false,
198       error: getErrorMessage(error, 'Unable to save transcript entry.'),
199     });
200   }
201 }

```

17. Transcript Table and Mark Normalization

Source: supabase/007_transcript_first_academic_schema.sql:29-66

- Defines the canonical transcript table: student, term, course, final_grade, status, attempt_no.
- The unique key prevents duplicate student/course/term/attempt records.

Code snippet

```

29 create table if not exists public.student_transcript_entries (
30   id uuid primary key default gen_random_uuid(),
31   student_id uuid not null references public.app_users(id) on delete cascade,
32   term_code text not null,
33   course_id uuid not null references public.courses(id) on delete restrict,
34   final_grade numeric(5,2),
35   status text not null check (status in ('passed', 'failed', 'withdrawn', 'in_progress')),
36   attempt_no integer not null default 1 check (attempt_no >= 1),
37   created_at timestampz not null default timezone('utc', now()),
38   updated_at timestampz not null default timezone('utc', now()),
39   unique (student_id, course_id, term_code, attempt_no)
40 );
41

```

```

42 create index if not exists idx_academic_terms_year_name
43   on public.academic_terms(academic_year, term_name);
44
45 create index if not exists idx_student_transcript_entries_student_term
46   on public.student_transcript_entries(student_id, term_code);
47
48 create index if not exists idx_student_transcript_entries_student_course
49   on public.student_transcript_entries(student_id, course_id, attempt_no desc, term_code desc);
50
51 drop trigger if exists trg_student_transcript_entries_updated_at on public.student_transcript_entries;
52 create trigger trg_student_transcript_entries_updated_at
53 before update on public.student_transcript_entries
54 for each row execute function public.set_updated_at();
55
56 create or replace function public.normalize_transcript_mark(input_mark numeric)
57 returns numeric
58 language sql
59 immutable
60 as $$
61   select case
62     when input_mark is null then null
63     when input_mark < 35 then 35
64     else input_mark
65   end
66   $$;

```

18. Transcript and GPA-derived Views

Source: supabase/007_transcript_first_academic_schema.sql:215-330

- student_transcript_v exposes joined transcript rows to the app.
- student_dashboard_summary_v derives GPA and completed credits from transcript marks for dashboards.

Code snippet

```

215 create or replace view public.student_transcript_v as
216 select
217   ste.id,
218   ste.student_id,
219   ste.term_code,
220   coalesce(at.term_type, public.term_type_from_term_name(public.term_name_from_code(ste.term_code))) as term_type,
221   c.course_code,
222   c.title as course_name,
223   c.credits,
224   ste.final_grade,
225   ste.status,
226   ste.attempt_no
227 from public.student_transcript_entries ste
228 join public.courses c on c.id = ste.course_id
229 left join public.academic_terms at on at.term_code = ste.term_code;
230
231 create or replace view public.student_term_metrics_v as
232 select
233   ste.student_id,
234   ste.term_code,
235   coalesce(at.term_type, public.term_type_from_term_name(public.term_name_from_code(ste.term_code))) as term_type,
236   count(*)::integer as course_count,
237   coalesce(sum(case when ste.status = 'passed' then c.credits else 0 end), 0)::integer as completed_credits,
238   round((avg(public.normalize_transcript_mark(ste.final_grade)) * 4 / 100.0)::numeric, 2) as gpa
239 from public.student_transcript_entries ste
240 join public.courses c on c.id = ste.course_id
241 left join public.academic_terms at on at.term_code = ste.term_code

```

```

242 group by
243     ste.student_id,
244     ste.term_code,
245     coalesce(at.term_type, public.term_type_from_term_name(public.term_name_from_code(ste.term_code)));
246
247 create or replace view public.student_dashboard_summary_v as
248 with transcript_marks as (
249     select
250         ste.student_id,
251         round((avg(public.normalize_transcript_mark(ste.final_grade)) * 4 / 100.0)::numeric, 2) as gpa
252     from public.student_transcript_entries ste
253     where ste.final_grade is not null
254     group by ste.student_id
255 ),
256 completed_credit_totals as (
257     select
258         passed.student_id,
259         coalesce(sum(c.credits), 0)::integer as completed_credits
260     from (
261         select distinct student_id, course_id
262         from public.student_transcript_entries
263         where status = 'passed'
264     ) passed
265     join public.courses c on c.id = passed.course_id
266     group by passed.student_id
267 ),
268 latest_draft as (
269     select distinct on (student_id)
270         id,
271         student_id,
272         name,
273         term_code,
274         status,
275         saved_at
276     from public.schedule_drafts
277     order by student_id, saved_at desc
278 ),
279 latest_evaluation as (
280     select distinct on (student_id)
281         id,
282         schedule_id,
283         student_id,
284         total_score,
285         risk_label,
286         total_credits,
287         recommendations,
288         explanation,
289         evaluated_at
290     from public.schedule_evaluations
291     order by student_id, evaluated_at desc
292 )
293 select
294     student_user.id as student_id,
295     student_user.university_id,
296     student_user.full_name as student_name,
297     coalesce(transcript_marks.gpa, 0) as gpa,
298     coalesce(completed_credit_totals.completed_credits, 0) as completed_credits,
299     sp.admission_year,
300     sp.admission_term,
301     d.name as department_name,

```

```

302     advisor_user.id as advisor_id,
303     advisor_user.full_name as advisor_name,
304     latest_draft.id as latest_draft_id,
305     latest_draft.name as latest_draft_name,
306     latest_draft.term_code as latest_draft_term_code,
307     latest_draft.status as latest_draft_status,
308     latest_draft.saved_at as latest_draft_saved_at,
309     latest_evaluation.id as latest_evaluation_id,
310     latest_evaluation.total_score as latest_total_score,
311     latest_evaluation.risk_label as latest_risk_label,
312     latest_evaluation.total_credits as latest_total_credits,
313     latest_evaluation.recommendations as latest_recommendations,
314     latest_evaluation.explanation as latest_explanation,
315     latest_evaluation.evaluated_at as latest_evaluated_at
316 from public.student_profiles sp
317 join public.app_users student_user on student_user.id = sp.user_id
318 join public.departments d on d.id = sp.department_id
319 left join public.app_users advisor_user on advisor_user.id = sp.advisor_id
320 left join transcript_marks on transcript_marks.student_id = sp.user_id
321 left join completed_credit_totals on completed_credit_totals.student_id = sp.user_id
322 left join latest_draft on latest_draft.student_id = sp.user_id
323 left join latest_evaluation on latest_evaluation.student_id = sp.user_id;
324
325 create or replace view public.advisor_advisee_summary_v as
326 select
327     summary.*,
328     public.get_dashboard_risk_status(summary.latest_total_score, summary.gpa) as risk_status
329 from public.student_dashboard_summary_v summary
330 where summary.advisor_id is not null;

```